BEGINNER-FRIENDLY PDF EDITION

Edit Contract Handbook

How to edit taxonomy, projections, routing, and database compatibility without touching release JSON.

Core rule: the taxonomy defines the universe; projections select from that universe; Verify proves that the edited release is still valid.

Prepared from `EDIT_CONTRACT_HANDBOOK.md`. Revised for readability, clean tables, and rendered flowcharts.

Contents

Recommended first pass for new users: read sections 1-6, then use the recipe that matches your edit.

1. First read: the mental model

A Semantic Release is a contract between the semantic model and a corpus database. It tells the system which concepts exist, which projections may use them, how documents should be routed, and what extracted values may become canonical database fields.

> **One sentence to remember**
> The taxonomy is the allowed universe; a projection is a selected view of that universe; Verify proves that the selection is still internally consistent.

Figure 1. The minimum mental model: taxonomy -> projection -> verification -> database decision.

The shortest useful workflow

1. Load an existing Semantic Release as a copy. Never start by overwriting the original.
2. Edit the taxonomy first. The taxonomy defines the values that may exist.
3. Refresh projection choices after taxonomy changes so pickers use the current taxonomy.
4. Edit projections by selecting taxonomy-defined values. Do not invent codes inside projections.
5. Update routing, markers, semantic bindings, and promotion rules wherever the changed concept is used.
6. Run Verify. Treat Verify as the gate, not as a formality.
7. Use the verification result to decide whether the current database can be updated, refilled, or must be materialized again.
8. Write or publish the draft only after Verify passes and the database decision is understood.

Why the contract matters

A valid release is not just a list of labels. It is a referential contract. Every projection must point back to real taxonomy entries. Every routing hint must use known concepts. Every promotion rule must target valid slots and valid source codes. Verification rebuilds the release from the edited copy and checks that these references still hold.

> **Do not edit release JSON during normal work**
> The Edit Suite exists to keep the release internally consistent while you tune it. Raw JSON edits should be reserved for exceptional maintenance work by people who understand the full contract.

2. Terms you need before editing

This handbook uses technical terms, but the editing logic is simple: define allowed concepts once, select them into projections, and verify all references before using the result.

Term	Plain meaning	Why it matters
Semantic Release	The versioned contract between the semantic model and a corpus database.	It tells the runtime which concepts are valid and how documents should be processed.
Taxonomy	The master vocabulary of the release.	A projection may select from it, but may not invent values outside it.
Taxonomy code	A stable machine key such as `invoice_total`.	Changing a code can break references and database compatibility.
Label / description / alias	Human-facing wording for a taxonomy item.	Use these for wording improvements instead of renaming machine codes.
Domain	A high-level semantic or business area.	Domains anchor document types, categories, fields, rows, cells, and projections.
Document type	A known kind of document the system can classify.	Projection routing often starts from document types.
Category / subcategory	Broad and more precise groups of extracted or detected information.	They constrain what belongs under a document type or projection.
Field code	A document-level extraction key.	Example: a total amount, contract date, or invoice number.
Row type	A repeated structure, often from a table or list.	Example: invoice line item, clause row, payment schedule row.
Cell code	A value inside a row type.	Cells and row types must make sense together.
Entity type	A reusable semantic object kind.	Example: organization, person, account, address.
Role type	A role played by an entity or party.	Example: buyer, seller, issuer, recipient, vendor.
Relation type	A permitted semantic link between concepts.	Relations must reference known concepts.
Projection	A controlled subset of the taxonomy for a workflow or document family.	It narrows the release so extraction and routing stay focused.
Routing	Hints used to choose a projection at runtime.	Examples, text markers, section roles, and party roles must remain consistent.
Promotion slot	A canonical database target slot.	Promotion slots are database-facing contract, not just labels.
Promotion rule	A rule that maps extracted values into promotion slots.	It must reference valid source codes and a valid target slot.
Verify	The consistency and compatibility check.	A release is not usable until Verify passes.
Refill	Reprocessing or refreshing stored semantic data in the same database.	Needed when the database can remain, but derived results must be updated.
Materialize new DB	Create a new materialized database for the edited release.	Needed when the semantic contract changed too much for a safe in-place update.

3. Safe workflow

The safe workflow is designed to avoid silent breakage. It keeps the original release intact, edits the vocabulary before the views, and lets verification decide database compatibility.

Figure 2. Safe edit order. Follow this order unless you have a specific reason not to.

What each step protects

Load a copy

Editing a copy gives you a safe workspace. The original release remains available if the edit needs to be abandoned or reworked.

Edit taxonomy first

Projections can only select values that exist in the taxonomy. If the vocabulary is missing, projection edits become workarounds.

Refresh choices

After taxonomy changes, picker options must be rebuilt from the current taxonomy copy. Otherwise new values may not appear where they should.

Verify before writing

Verify is the gatekeeper. It rebuilds the release, checks references, and reports the database action.

Before you edit, answer three questions

1. Is this only a wording change, or does it change the machine contract?
2. Which projections should use the changed concept?
3. Could this change affect stored database results, promotion slots, or projection IDs?

> **Practical default**
> If you only want better wording, change labels, descriptions, aliases, or routing text. Do not rename codes.

4. Taxonomy in plain language

The taxonomy is the master vocabulary of a release. It defines the complete set of semantic concepts the system is allowed to know for that release.

Think of the taxonomy as the source of truth. A projection may narrow the taxonomy, but it may not invent concepts. If a value is not in the taxonomy, it cannot be selected into a projection.

Typical taxonomy sections

Section	What it defines	When users touch it
`domains`	High-level semantic or business areas.	Use when a concept belongs to a broad area such as finance, contracts, HR, or operations.
`document_types`	Document classes the system may recognize.	Use for document routing and classification.
`categories`	Broad groups of information.	Use to organize extraction and semantic coverage.
`subcategories`	More precise groups below categories.	Use when a category needs a valid child concept.
`field_codes`	Document-level fields.	Use for single values extracted from a document.
`row_types`	Repeated structures.	Use for tables, schedules, line items, and repeated clauses.
`cell_codes`	Values inside row types.	Use together with row types, not as isolated concepts.
`entity_types`	Canonical entity kinds.	Use when values represent entities that may be materialized or linked.
`role_types`	Roles played by entities or parties.	Use for party semantics and routing roles.
`relation_types`	Allowed semantic relationships.	Use when entities or concepts need explicit links.
`promotion_slots`	Canonical database targets.	Use only for stable, database-facing values.

Why the taxonomy is structured

The taxonomy is structured instead of flat because each system task needs a different kind of concept. Classification needs document types. Extraction needs fields, rows, and cells. Routing needs domains, examples, markers, and roles. Database materialization needs stable promotion slots. Audit needs stable IDs and descriptions. The structure also lets Verify distinguish wording-only edits from structural changes.

4a. Taxonomy architecture and hierarchy

These diagrams replace the raw Mermaid blocks with rendered reference diagrams. Read each arrow as "depends on" or "must be checked when the source changes."

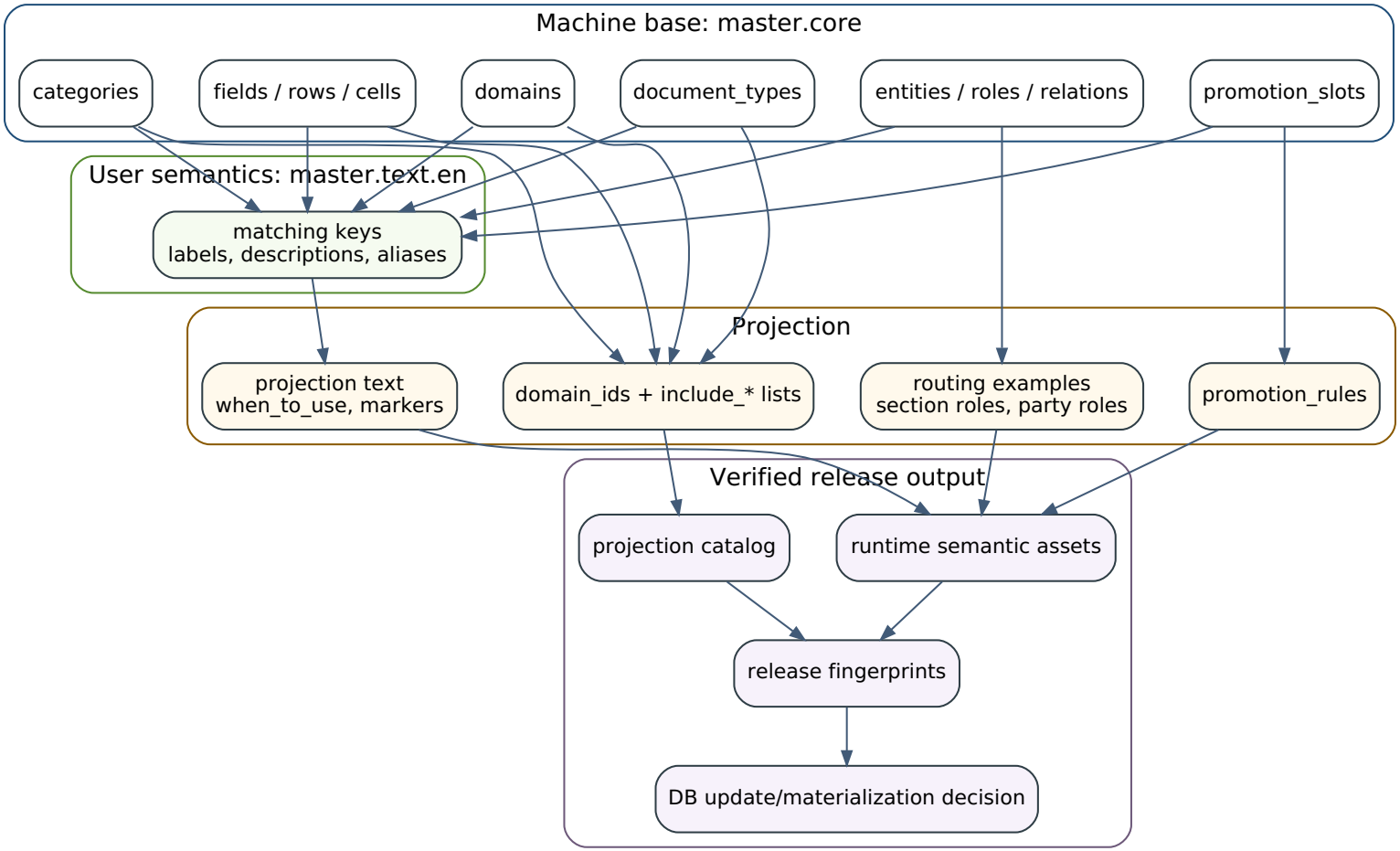

Figure 3. The release has a machine base, user-facing semantic text, projection selections, and verified runtime outputs.

4b. Hierarchy reference graph

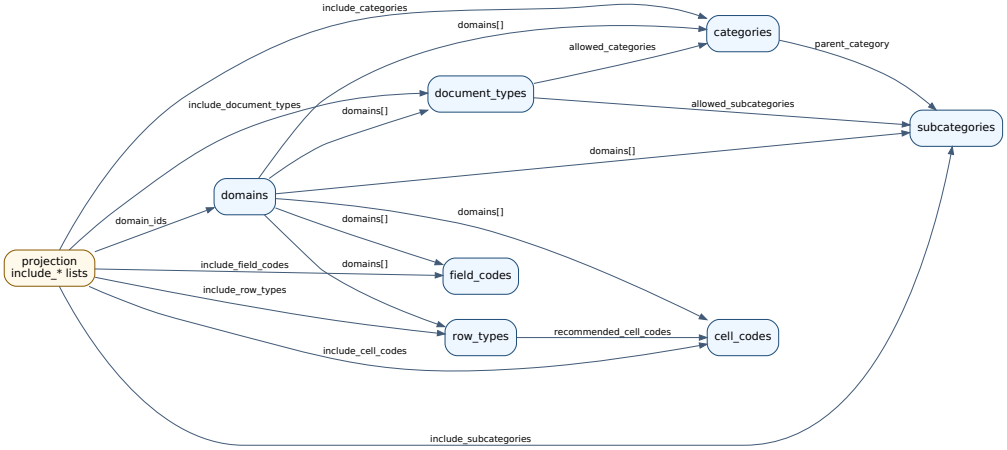

Figure 4. Classification and extraction hierarchy: domains, document types, categories, fields, rows, cells, and projection includes.

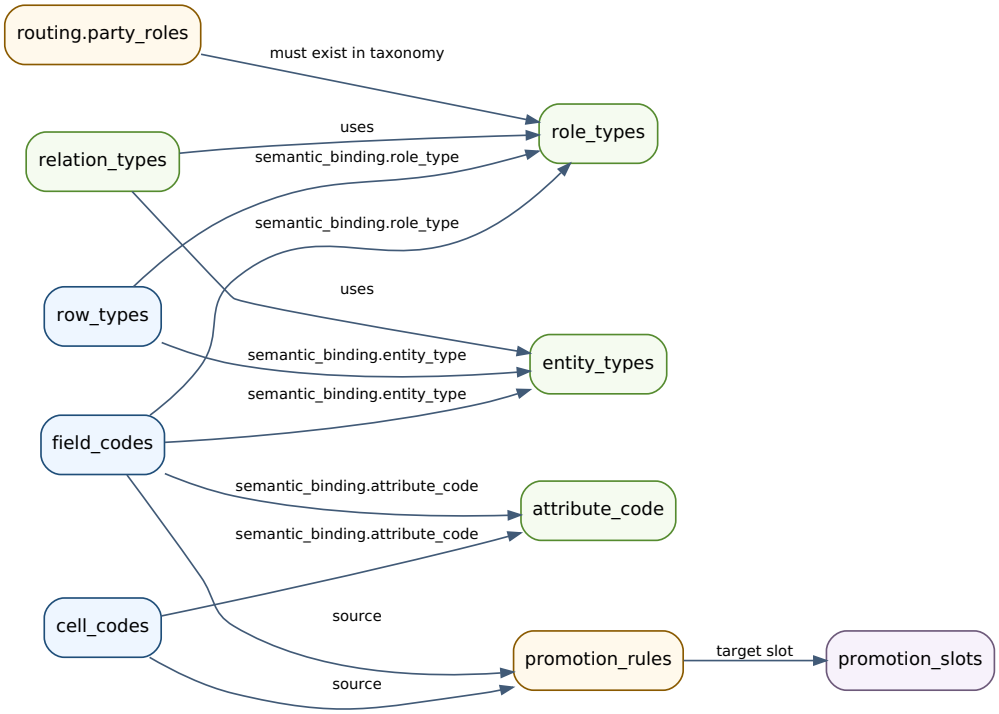

Figure 5. Semantic binding and promotion references: fields, rows, cells, roles, relations, and database slots.

The original hierarchy is not a simple tree. It is a layered graph: machine concepts are referenced by text, projections, routing, semantic bindings, promotion rules, and database compatibility checks.

5. Machine contract vs user semantics

Every taxonomy concept has two planes: a machine plane and a user semantic plane. The machine plane is the contract. The user semantic plane explains and tunes that contract.

Machine plane

Stable IDs, codes, references, constraints, bindings, value types, projection includes, promotion slots, and rule paths.

User semantic plane

Labels, descriptions, aliases, explanatory text, routing wording, text markers, and domain markers.

Layer	Machine contract	User semantics	Rule
Release identity	`release_id`, `release_version`, fingerprints, and master taxonomy release ID.	Description or notes, if present.	Identity changes can affect database compatibility.
Taxonomy item key	Map key or `code`, for example `invoice_total`.	Label such as *Total Amount Due*.	Prefer changing labels over changing codes.
Taxonomy core	`status`, `domains`, `parent_category`, constraints, `value_type`.	None inside core.	Core should not carry text-only authoring content.
Taxonomy text	Same keys as taxonomy core.	`label`, `description`, `aliases`.	Text keys must match core keys for every supported locale.
Semantic binding	`entity_type`, `role_type`, `attribute_code`, row materialization flags.	Explanation of what the binding means.	Binding references must point to valid machine concepts where validation requires it.
Projection core	`projection_id`, `domain_ids`, `include_*`, `extends`, routing references, `promotion_rules`.	None inside core.	Projection core may only select known taxonomy values.
Projection text	No structural coverage.	`label`, `description`, `when_to_use`, `avoid_when`, markers.	Text tunes routing and user clarity, but does not create taxonomy concepts.
Promotion	Slot IDs and rule source paths.	Slot labels or descriptions, if exposed.	Slots and source codes are database-facing contract.

Core/text mirror rule

For taxonomy sections, core and text must mirror each other by key. If `master.core.field_codes` contains `invoice_total`, then `master.text.en.field_codes` must also contain `invoice_total`. If core is missing a text key, users lack meaning. If text has an extra key, it describes a concept that does not exist.

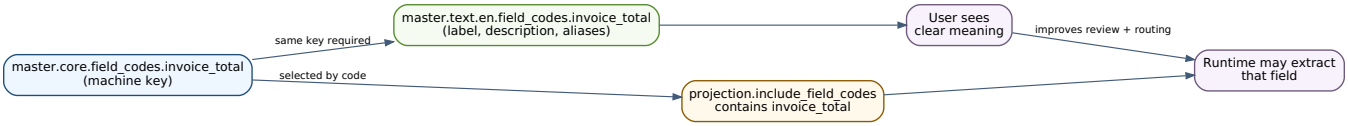

Figure 6. Core and text use the same taxonomy key; projections select the machine key; user text gives the key meaning.

Important nuance
User semantics can still affect behavior. Aliases help lookup. Text markers help routing. Bad wording can make the system behave worse. But user semantics do not create new machine concepts.

Simple rule for wording changes

When a concept keeps the same meaning, keep the code and improve the label or description. For example, keep `invoice_total` and change the label from *Invoice Total* to *Total Amount Due*. Rename the code only when you are deliberately changing the machine contract and will update every reference.

6. Projections in plain language

A projection is a controlled subset of the taxonomy. It defines which taxonomy concepts are active for one semantic view, workflow, document family, domain, or extraction context.

Why projections exist

The master taxonomy may be broad. A single release can cover many document types, domains, and extraction patterns. The runtime should not always load or reason over the full taxonomy for every document. Projections narrow the release so routing and extraction stay focused.

For example, a contract projection may include contract document types, party roles, legal fields, and notice sections. An invoice projection may include invoice document types, vendor/customer roles, line item rows, tax cells, and payment fields. Both can share taxonomy entries, but each exposes only the subset it needs.

What a projection usually contains

Projection part	Purpose	Rule of thumb
`projection_id`	Stable ID of the projection.	Do not change for wording only.
`label` and `description`	Human-readable purpose.	Make these clear enough for a new user to choose the projection.
`domain_ids`	Domains this projection belongs to.	Must reference taxonomy domains.
`include_document_types`	Document types active in this projection.	Routing examples should be a subset of this list.
`include_categories` / `include_subcategories`	Information groups active in this projection.	Do not promise coverage in text that is not selected here.
`include_field_codes`	Document-level fields active in this projection.	Promotion rules must not reference removed fields.
`include_row_types` / `include_cell_codes`	Repeated structures and their cells.	Rows and cells should be edited together.
`routing`	Runtime hints for choosing the projection.	Use real document types, valid roles, and specific markers.
`promotion_rules`	Optional mappings into database slots.	Advanced. Use only when source codes and slots are valid.

Projection picker rules

Projection editors should use controlled selectors. Document type options come from taxonomy document types; category options from taxonomy categories; fields from taxonomy field codes; rows from row types; cells from cell codes; party roles from role types.

If the picker does not show the value you need
The value is not available to that projection yet. Add or fix it in the taxonomy, refresh projection choices, then select it. Do not work around the picker by writing raw JSON.

Good projection behavior

- It includes the document types it claims to handle.
- It includes the categories, fields, rows, and cells needed for those document types.
- Its routing examples are real document types included by that same projection.
- Its party roles are backed by taxonomy role types.
- Its text markers are specific enough to help routing without causing false matches.
- Its promotion rules reference only included or valid source codes and valid promotion slots.

7. Change propagation reference

Use this table before Verify. It tells you what must be updated or checked when a base taxonomy item changes.

Change	Must update or check	Why
Add a domain	Core domain entry; matching text entry; `domains[]` on document types, categories, subcategories, fields, rows, and cells; projection `domain_ids`; domain markers.	Domains anchor taxonomy coverage and projection routing.
Rename a domain code	Every `domains[]` reference; projection `domain_ids`; domain markers; text key.	A domain code is a machine reference, not display wording.
Delete a domain	Remove or replace all `domains[]` references and projection `domain_ids` first.	Nothing may point to a deleted domain.
Add a document type	Text entry; domain assignment; allowed categories/ subcategories; projection `include_document_types`; routing examples; text markers.	Classification and routing must both know the new type.
Rename a document type code	Projection includes; routing examples; stored routing assumptions; text key.	Example document types must remain valid and included.
Delete a document type	Projection includes and routing examples.	A projection cannot route by a missing document type.
Add a category	Text entry; domain assignment; document type `allowed_categories`; projection `include_categories`.	Categories constrain classification and projection coverage.
Rename or delete a category	Subcategory `parent_category`; document type `allowed_categories`; projection `include_categories`; text key.	Subcategories and document type constraints depend on category codes.
Add a subcategory	Text entry; valid `parent_category`; domain assignment; document type `allowed_subcategories`; projection `include_subcategories`.	Subcategories are children of categories and may be constrained by document types.
Rename or delete a subcategory	Document type `allowed_subcategories`; projection `include_subcategories`; text key.	Document type constraints and projection coverage must stay valid.
Add a field code	Text entry; `value_type`; `semantic_binding`; projection `include_field_codes`; promotion rules if needed.	Fields define allowed document-level extraction keys.
Rename or delete a field code	Projection `include_field_codes`; promotion rule source paths; semantic bindings; text key.	Extraction output and database promotion may depend on that field.
Add a row type	Text entry; domain assignment; `recommended_cell_codes`; `semantic_binding`; projection `include_row_types`.	Rows define repeated structures and their entity semantics.
Rename or delete a row type	Projection `include_row_types`; row semantic assumptions; text key.	Structured rows cannot use unknown row types.

Change	Must update or check	Why
Add a cell code	Text entry; `value_type`; `semantic_binding`; row `recommended_cell_codes`; projection `include_cell_codes`; promotion rules if needed.	Cells define allowed values inside rows.
Rename or delete a cell code	Row `recommended_cell_codes`; projection `include_cell_codes`; promotion rule source paths; text key.	Rows and promotion rules may reference cells directly.
Add an entity type	Text entry; field or row `semantic_binding.entity_type`; relation definitions if used.	Entity types give fields and rows their materialization target.
Rename or delete an entity type	Field and row semantic bindings; relation definitions; text key.	Bindings must not point to missing entity types.
Add a role type	Text entry; field, row, cell, or relation bindings if needed; projection `routing.party_roles`.	Roles affect routing and semantic binding.
Rename or delete a role type	Bindings and projection `party_roles`; text key.	Party roles and role bindings must stay valid.
Add a relation type	Text entry and valid referenced concepts.	Relations describe allowed semantic links.
Rename or delete a relation type	Relation references and text key.	Relation semantics must remain explicit.
Add a promotion slot	Slot definition; value type; promotion rules; database compatibility review.	Promotion slots are canonical database targets.
Rename or delete a promotion slot	Every promotion rule targeting that slot; database compatibility review.	Slot IDs are database-facing contract.

8. Projection change reference

Projection edits are safe only when the selected taxonomy values, routing hints, and promotion rules still agree with each other.

Projection change	Check this	Why it matters
Change `domain_ids`	Projection still includes document types from that domain; routing text still describes the projection; unrelated document types were not left behind.	A projection that says it belongs to one domain but routes another will behave unpredictably.
Change document type coverage	`routing.example_document_types` is a subset of `include_document_types`; markers still match the included types; fields, rows, and cells still fit.	Routing examples that are not included should fail verification.
Change category/subcategory coverage	Included subcategories have valid parent categories; included fields still belong to meaningful categories; text does not promise removed coverage.	Coverage text and selected taxonomy must agree.
Change field coverage	Promotion rules do not reference removed fields; semantic bindings still make sense; expected extraction output still matches the workflow.	Removing a field can break promotion or downstream expectations.
Change row/cell coverage	Rows and cells still make sense together; recommended cells exist in taxonomy; projections include the cells needed to interpret included rows; promotion rules do not reference removed cells.	Rows without needed cells are usually incomplete.
Change routing	Examples are real included document types; section roles are known; party roles come from taxonomy role types; markers are specific; `avoid_when` does not contradict `when_to_use`.	Routing should help choose a projection. It should not become a second taxonomy.

Routing checklist

Routing may include `when_to_use`, `avoid_when`, `text_markers`, `example_document_types`, `section_roles`, and `party_roles`. When changing routing, make sure examples are real included document types, section roles are known, party roles are selected from taxonomy role types, markers are specific, and `avoid_when` does not contradict `when_to_use`.

> **Routing should not become a second taxonomy**
> Routing tells the system when to choose a projection. It should not introduce new concepts that are missing from the taxonomy.

9. Practical editing recipes

Use the recipe that matches your edit. Then run Verify. Even safe edits must pass verification before the release is used.

Recipe	Steps	Watch out for
Improve wording only	Edit labels, descriptions, aliases, `when_to_use`, `avoid_when`, and markers.	Do not change codes, IDs, include lists, bindings, or promotion rules. Verify afterwards.
Add a document type	Add taxonomy entry; assign domains and constraints; refresh projection choices; include it in the right projection; add routing examples/ markers; verify coverage.	A document type in taxonomy alone is not enough. At least one projection must include and route it.
Add an extraction field	Add field code; define label, description, value type, and binding; refresh choices; include field in projections; update promotion rules only if needed; verify.	Do not assume extraction will use a new field unless it is selected into the relevant projections.
Add row and cell coverage	Add or update row type; add needed cell codes; connect cells with `recommended_cell_codes` ; include both row and cells in projections; verify.	Rows and cells should be changed as a set.
Add a role type	Add taxonomy role; describe it clearly; refresh choices; select it in `routing.party_roles` where relevant; update semantic bindings if needed; verify.	A role type is a role, not a document type, field, or category.
Narrow a projection	Remove unrelated document types and taxonomy selections; repair routing examples/markers; check promotion rules; verify.	A narrower projection should be easier to route and extract, not merely smaller.
Remove a concept	Find every reference; remove from projections, routing, bindings, row recommendations, and promotion rules; remove or replace child concepts; delete taxonomy entry; verify.	Deletion is valid only when nothing points to the deleted concept.

Add-and-migrate beats risky renaming

If a concept is truly changing meaning, prefer adding a new code and deliberately migrating projections to it. Remove the old code only after no projection, promotion rule, semantic binding, relation, row recommendation, document type constraint, or database mapping needs it.

Deletion rule

Deletion is valid only when nothing still points to the deleted concept. Before deleting a taxonomy code, search for it in projection include lists, routing examples and party roles, row recommended cell codes, semantic bindings, relation definitions, promotion rules, and document type constraints.

10. Verification and troubleshooting

Verify is the gatekeeper. It rebuilds the edited copy as a release and checks whether the contract still holds.

What Verify may check

- normalization of the master taxonomy shape and projection payloads
- projection IDs and projection catalogs
- runtime semantic assets and release fingerprints
- taxonomy references, projection coverage, routing consistency, and promotion rules
- database compatibility against the selected current database

Common verification failures

Failure	Meaning	Fix
Unknown code	A projection, rule, binding, row recommendation, or routing hint points to a taxonomy code that does not exist.	Add the missing taxonomy entry, or replace/remove the reference.
Example document type is not included	A projection uses a document type as a routing example, but that type is not in `include_document_types`.	Include the document type in that projection, or remove it from the examples.
Projection has too little coverage	The projection includes document types but lacks supporting categories, fields, rows, cells, or roles.	Add the taxonomy selections needed to make the projection operational.
Invalid section role	Routing uses a section role outside the known role list.	Select a known section role. Do not invent section roles in free text.
Invalid party role	Routing uses a party role that is not backed by taxonomy role types.	Add the role type to the taxonomy, or select an existing one.
Invalid promotion rule	A promotion rule points to a missing promotion slot or missing source code.	Correct the target slot, source field, source row, or source cell reference.
Core/text mismatch	A taxonomy key exists in core but not in text, or text describes a key that core does not contain.	Add the missing text entry, remove the orphan text entry, or repair the key name.
DB compatibility cannot be decided	The release was verified without a current database path.	Select the current database and run Verify again.

How to troubleshoot quickly

1. Copy the failing code exactly from the Verify result.
2. Search for that code in taxonomy core, taxonomy text, projection include lists, routing, row recommendations, semantic bindings, relation definitions, and promotion rules.
3. Decide whether the correct fix is to add the missing taxonomy entry or remove/replace the invalid reference.

4. Refresh projection choices if taxonomy changed.
5. Run Verify again.

> **A failed Verify is useful information**
> It does not mean the edit was bad. It means the edited copy still contains unresolved references or compatibility questions.

11. Database update decisions

After verification, the system determines whether the current database can be updated, needs refill, or requires a new materialized database. Do not guess this manually.

Decision	Meaning	Typical case
Update current DB	The edited release is compatible with the current database.	Likely for text-only or narrow non-breaking edits. Metadata and release assets can be updated without reprocessing the corpus.
Update current DB with auto refill	The database can remain, but stored semantic data or derived results need to be refreshed.	Likely when projection coverage changed but the master release line remains compatible.
Materialize new DB	The release contract changed too much for safe in-place update.	Expected for incompatible taxonomy release changes, missing active projection IDs, stored documents that point to removed projections, or structural changes that alter stored semantics.
Select current DB	Compatibility cannot be decided because no current database was selected.	Select the database path and verify again.

Why materializing a new database is not a failure

Materializing a new database is the correct outcome when the semantic contract changed too much for a safe in-place update. This can happen when the master taxonomy release ID changes incompatibly, active documents were processed under another taxonomy line, required projection IDs are missing, stored documents point to projections not present in the edited release, or structural taxonomy changes alter the meaning of stored semantic data.

12. Final checklist and anti-patterns

Use this page before writing or publishing the edited copy.

Quality checklist

Area	Confirm
Release safety	Release was loaded as a copy; original was not overwritten; raw JSON was not edited for normal work.
Taxonomy correctness	Taxonomy edits are intentional; codes are stable; text keys mirror core keys; descriptions are clear to a new user.
Projection correctness	Projection choices were refreshed; projections use only taxonomy-defined values; document type coverage is useful; routing examples are included by the same projection.
Routing correctness	Party roles come from taxonomy role types; section roles are known; text markers are specific; `when_to_use` and `avoid_when` do not contradict each other.
Rows, cells, and promotion	Row and cell coverage is consistent; promotion rules target valid slots and source codes; removed source codes are not still referenced.
Verification and database	Verify passes; database decision is understood before writing or publishing the draft.

Anti-patterns to avoid

Anti-pattern	Better approach
Editing raw JSON for normal taxonomy or projection work	Use the Edit Suite controls. Raw JSON bypasses the consistency protections.
Typing invented codes into projections	Add the concept to the taxonomy first, then refresh projection choices.
Changing a stable code just to improve wording	Change the label or description instead.
Adding taxonomy entries without adding them to projections	The taxonomy allows the concept; projections decide where it is active.
Adding document types without routing hints	The runtime needs enough evidence to choose the right projection.
Adding rows without the cells needed to interpret them	Rows and cells are a pair.
Adding promotion rules before source codes and slots exist	Promotion rules must point to known source codes and target slots.
Ignoring Verify warnings	Verification is the gate that proves the release is usable.

Final rule

The whole handbook in one line
The taxonomy defines the universe. Projections select from that universe. Verification proves that the edited universe and its selected views still form a valid release.